

Python: características, tipos de datos y operadores básicos

Python es un lenguaje de programación de alto nivel. Un lenguaje de programación es un sistema formal que permite expresar instrucciones de manera precisa para que un ordenador pueda ejecutarlas. A través de un lenguaje de programación, el ser humano traduce una idea, un problema o un procedimiento en una secuencia de órdenes comprensibles para la máquina.

En este sentido, Python actúa como un puente entre el pensamiento humano y el funcionamiento interno del ordenador: permite representar datos, aplicar operaciones, tomar decisiones y organizar procesos de forma estructurada. Con Python podemos crear programas, automatizar tareas, analizar datos, construir aplicaciones web, trabajar con inteligencia artificial o resolver problemas matemáticos.

1. Qué es Python

Python fue creado por Guido van Rossum a finales de los años 80 y publicado por primera vez en 1991. Nació con una intención muy clara: crear un lenguaje potente, pero fácil de leer y de escribir.

En estos apuntes trabajaremos con Python 3.14.7, que es la última versión estable de Python 3 publicada actualmente. Usar una versión reciente permite disponer de las mejoras y correcciones más actuales del lenguaje.

Su influencia principal viene de un lenguaje llamado ABC, que estaba pensado para enseñar programación de forma sencilla. Python también recibió ideas de otros lenguajes, como C, Modula-3, Lisp, Perl, Unix Shell y Smalltalk. Por eso combina sencillez, potencia, facilidad para automatizar tareas y una visión muy orientada a objetos.

La idea más importante de Python es que el código debe ser legible. Un programa no solo lo lee el ordenador; también lo leen otras personas, incluido el propio programador semanas o meses después.

2. Características principales

Característica	Explicación sencilla
Alto nivel	Permite programar sin manejar directamente detalles internos del ordenador.
Interpretado	Normalmente se ejecuta mediante un intérprete, lo que facilita probar código rápidamente.
Legible	Su sintaxis busca parecer clara y ordenada.
Tipado dinámico	No es necesario declarar previamente el tipo de dato.
Tipado fuerte	No mezcla tipos incompatibles sin que el programador lo indique.
Multiparadigma	Permite programar de varias formas: imperativa, funcional y orientada a objetos.
Gran ecosistema	Tiene muchas bibliotecas para datos, IA, web, ciencia, automatización y educación.

```
nombre = "Ana"
edad = 16

print("Hola", nombre)
print("Tienes", edad, "años")
```

Idea clave: Python intenta que el código sea sencillo de escribir y sencillo de leer.

3. Datos y tipos de datos

Un programa trabaja con datos. Un dato puede ser un número, un texto, una respuesta verdadera o falsa, una lista de elementos o una fecha. Cada dato tiene un tipo. El tipo indica qué clase de información es y qué operaciones podemos hacer con ella.

```
nombre = "Laura"
edad = 20
altura = 1.70
aprobado = True

print(type(nombre))
print(type(edad))
print(type(altura))
print(type(aprobado))
```

La función `type()` permite consultar el tipo de un dato. Es muy útil al empezar, porque ayuda a comprobar qué está manejando realmente el programa.

4. Clasificación general de tipos de datos

Una clasificación útil para principiantes es distinguir entre datos simples y datos compuestos.

Datos simples

Representan un valor individual: un número, un texto, un verdadero/falso o la ausencia de valor.

Tipo	Nombre en Python	Ejemplo
Entero	int	25
Decimal	float	1.75
Texto	str	"Hola"
Booleano	bool	True / False
Sin valor	NoneType	None
Complejo	complex	3 + 2j

Datos compuestos

Agrupan varios valores en una sola estructura.

Tipo	Nombre en Python	Ejemplo
Lista	list	[1, 2, 3]
Tupla	tuple	(1, 2, 3)
Diccionario	dict	{"nombre": "Ana", "edad": 16}
Conjunto	set	{1, 2, 3}
Conjunto inmutable	frozenset	frozenset({1, 2, 3})
Rango	range	range(1, 10)

Datos binarios

Tipo	Nombre en Python	Uso general
Bytes	bytes	Datos binarios que no se modifican.
Array de bytes	bytearray	Datos binarios modificables.
Vista de memoria	memoryview	Acceso a datos binarios sin copiarlos.

Fechas y tiempo

Python no tiene un tipo de fecha básico como int o str. Para trabajar con fechas se usa normalmente el módulo datetime.

Tipo	Módulo	Uso
Fecha	datetime.date	Día, mes y año.
Hora	datetime.time	Hora, minutos y segundos.
Fecha y hora	datetime.datetime	Fecha completa con hora.
Duración	datetime.timedelta	Diferencia entre fechas u horas.
Zona horaria	datetime.timezone	Información de zona horaria.

```
from datetime import date

hoy = date.today()
print(hoy)
```

5. Tipos simples explicados

int: números enteros

El tipo int representa números enteros, positivos o negativos, sin decimales.

```
edad = 16
alumnos = 25
temperatura = -3
```

Se usa para contar elementos, guardar edades, puntuaciones, cantidades o posiciones.

float: números decimales

El tipo float representa números con decimales. Se usa para medidas, precios, notas medias o porcentajes.

```
altura = 1.75
precio = 19.99
nota_media = 8.5
```

Los decimales en informática pueden tener pequeñas imprecisiones. Para principiantes basta con saber que float sirve para números con parte decimal.

str: texto

El tipo str representa texto. Un texto puede contener letras, números escritos como texto, espacios y símbolos.

```
nombre = "María"
ciudad = "Valencia"
mensaje = "Hola, buenos días"
```

Un número escrito entre comillas no se comporta como número, sino como texto.

```
edad_numero = 16
edad_texto = "16"

print(edad_numero + 1)      # 17
print(edad_texto + " años") # 16 años
```

bool: verdadero o falso

El tipo bool solo tiene dos valores posibles: True y False. Es fundamental para hacer comparaciones y tomar decisiones.

```
edad = 17
mayor_de_edad = edad >= 18
```

```
print(mayor_de_edad) # False
```

NoneType: ausencia de valor

En Python no se usa NULL. El valor que representa la ausencia de dato es None. Se escribe con mayúscula inicial.

Valor	Qué significa
0	Hay un número, y ese número es cero.
""	Hay un texto, pero está vacío.
False	Hay un valor lógico, y es falso.
None	No hay dato o todavía no se ha asignado un valor útil.

```
resultado = None

if resultado == None:
    print("Todavía no hay resultado")
```

Más adelante se suele usar una forma más técnica para comprobar None, pero para empezar es suficiente entender que None significa ausencia de valor.

complex: números complejos

Python también permite trabajar con números complejos, como $3 + 2j$. No son importantes en una primera etapa, salvo en contextos matemáticos o científicos.

```
z = 3 + 2j
print(z)
```

6. Operadores básicos

Los operadores permiten hacer operaciones con los datos. Para empezar, conviene estudiar solo los operadores básicos: aritméticos, comparativos, lógicos y de asignación.

Operadores aritméticos

Operador	Significado	Ejemplo	Resultado
+	Suma	$3 + 2$	5
-	Resta	$3 - 2$	1

*	Multiplicación	3 * 2	6
/	División decimal	5 / 2	2.5
//	División entera	5 // 2	2
%	Resto de la división	5 % 2	1
**	Potencia	2 ** 3	8

```

a = 7
b = 2

print(a + b)
print(a / b)
print(a // b)
print(a % b)

```

Operadores comparativos

Sirven para comparar valores. El resultado de una comparación siempre es True o False.

Operador	Significado	Ejemplo
==	Igual que	edad == 18
!=	Distinto de	edad != 18
>	Mayor que	edad > 18
<	Menor que	edad < 18
>=	Mayor o igual que	edad >= 18
<=	Menor o igual que	edad <= 18

```

edad = 17

print(edad >= 18)  # False
print(edad < 18)   # True

```

Operadores lógicos

Sirven para combinar condiciones.

Operador	Significado	Ejemplo
and	Y: deben cumplirse las dos condiciones	edad >= 18 and tiene_permiso
or	O: basta con que se cumpla una condición	es_profesor or es_alumno

not	Niega una condición	not aprobado
-----	---------------------	--------------

```
edad = 20
tiene_permiso = True

puede_entrar = edad >= 18 and tiene_permiso
print(puede_entrar)  # True
```

Operadores de asignación

Sirven para guardar o actualizar valores.

Operador	Ejemplo	Significado
=	x = 5	Guarda el valor 5 con el nombre x.
+=	x += 1	Suma 1 al valor actual.
-=	x -= 1	Resta 1 al valor actual.
*=	x *= 2	Multiplica el valor actual por 2.
/=	x /= 2	Divide el valor actual entre 2.

```
contador = 0

contador += 1
contador += 1

print(contador)  # 2
```

Para una primera unidad, estos operadores son suficientes. Los operadores más avanzados se pueden estudiar cuando ya se dominen las bases.